

Лабораторная работа № 12
“Установка и настройка Node.js и NPM”

Цель: “Сформировать умение анализировать методы работы таймеров.”

Ход работы

1. Подготовка (обновление основной ветки)

Перед началом работы убедитесь, что ваша локальная `main` ветка актуальна:

```
bash  
git checkout main  
git pull origin main
```

2. Создание новой ветки для заданий

Создайте ветку с названием `feature/event-loop-tasks`:

```
bash  
git checkout -b feature/event-loop-tasks
```

Все дальнейшие изменения будут выполняться в этой ветке.

3. Задание 1: Модернизация HTTP-сервера с событиями

Задача: Модифицируйте файл `index.js` так, чтобы ваш HTTP-сервер использовал событийную модель для обработки запросов.

Требования:

3.1. Создайте класс `AppServer`, который наследуется от `EventEmitter`.

3.2. В классе реализуйте:

- Метод `start(port)` — запускает HTTP-сервер и генерирует событие `'server:started'` с портом.
- Метод `stop()` — останавливает сервер и генерирует событие `'server:stopped'`.
- При входящем запросе генерируйте событие `'request:received'` с объектом запроса (`req.url` и `req.method`).

3.3. Зарегистрируйте обработчики:

- При `'server:started'` — выводить в консоль: 🚀 Сервер запущен на порту `<port>`
- При `'request:received'` — выводить: ✉ Получен запрос: `<method>` `<url>`
- При `'server:stopped'` — выводить: 🛑 Сервер остановлен

3.4. Сервер должен отвечать на все запросы текстом: "Hello from Event-Driven Server!"

Пример запуска в `index.js`:

```
javascript

const app = new AppServer();
app.start(3000);

// Эмуляция остановки через 10 секунд
setTimeout(() => {
  app.stop();
}, 10000);
```

4. Задание 2: Создание системы логирования через события (Среднее)

Задача: Реализовать модуль `logger.js`, который будет подписываться на события сервера и логировать их в файл.

Требования:

4.1. Создайте файл `logger.js` в корне проекта.

4.2. В `logger.js` экспортируйте функцию `setupLogger(app)`, которая:

- Подписывается на все события сервера (`server:started`, `server:stopped`, `request:received`).
- Записывает каждое событие в файл `logs.txt` в формате: `[ВРЕМЯ] СОБЫТИЕ: ДАННЫЕ`

4.3. Используйте модуль `fs` для записи в файл (асинхронно).

4.4. В `index.js` подключите и настройте логгер:

javascript

```
const logger = require('./logger');
```

```
logger.setupLogger(app);
```

Ожидаемый результат: После запуска и нескольких запросов в папке проекта появляется файл `logs.txt` с записями всех событий.

5. Задание 3: Асинхронная обработка данных с таймерами (Сложное)

Задача: Добавить в сервер эмуляцию асинхронной обработки заказов через события и таймеры.

Требования:

5.1. Создайте отдельный класс `OrderHandler` (в том же `index.js` или отдельном файле), наследующий `EventEmitter`.

5.2. Реализуйте метод `processOrder(orderId)`, который:

- Генерирует событие `'order:start'` с `orderId`.
- Через 2 секунды (используйте `setTimeout`) генерирует событие `'order:processing'` с текстом "Идёт обработка...".
- Ещё через 2 секунды генерирует событие `'order:complete'` с `orderId` и случайной суммой от 100 до 1000.

5.3. На событие `'order:complete'` повесьте обработчик, который:

- Вычисляет число Пи (π) с точностью до 7 знаков (3.1415926) без использования сторонних библиотек.
- Выводит в консоль: 💰 Заказ #<orderId> завершён на сумму <sum> руб. PI = <вычисленное_значение>

5.4. Добавьте эндпоинт в HTTP-сервер: при GET-запросе на `/order/<id>` запускайте обработку заказа.

Пример работы:

text

Запрос: GET /order/42

→ [order:start] Заказ #42 начат
→ Через 2 сек: [order:processing] Заказ #42: Идёт обработка...
→ Через 4 сек: [order:complete] Заказ #42 завершён на сумму 756 руб. PI = 3.1415926

6. Задание 4: Исследование цикла событий (Экспериментальное)

Задача: Написать скрипт `event-loop-demo.js`, демонстрирующий порядок выполнения различных асинхронных операций.

Требования:

6.1. Создайте файл `event-loop-demo.js`.

6.2. В файле последовательно вызовите:

- `setTimeout(() => {...}, 0)` с выводом "1. setTimeout"
- `setImmediate(() => {...})` с выводом "2. setImmediate"
- `process.nextTick(() => {...})` с выводом "3. process.nextTick"
- `Promise.resolve().then(() => {...})` с выводом "4. Promise.then"
- Синхронный `console.log("5. Синхронный код")`

6.3. Запустите скрипт и запишите порядок вывода.

6.4. В комментариях в коде напишите объяснение, почему вывод произошёл именно в таком порядке (со ссылкой на фазы цикла событий).

Ожидаемый вывод:

text

5. Синхронный код

3. process.nextTick

4. Promise.then

1. setTimeout

2. setImmediate

7. Задание 5: Создание собственного события с параметрами (Творческое)

Задача: Реализовать кастомное событие 'user:action', которое будет передавать сложный объект.

Требования:

7.1. Создайте класс `UserTracker`, наследующий `EventEmitter`.

7.2. Реализуйте метод `trackAction(userId, action, metadata)`, который:

- Генерирует событие 'user:action' с объектом:

javascript

```
{
  userId: userId,
  action: action,
  timestamp: new Date().toISOString(),
  metadata: metadata,
  id: Math.random().toString(36).substr(2, 9) // уникальный ID события
}
```

7.3. Зарегистрируйте обработчик, который выводит в консоль красиво отформатированное сообщение:

text

```
 Пользователь <userId> совершил действие "<action>"
  Время: <timestamp>
  ID события: <id>

  Доп. данные: <metadata>
```

7.4. Создайте несколько тестовых вызовов `trackAction` с разными параметрами.

8. Фиксация изменений и создание Pull Request

8.1. Добавьте все созданные и изменённые файлы в Git:

bash

```
git add index.js logger.js event-loop-demo.js package.json
```

```
git status # Проверьте, что все нужные файлы добавлены
```

8.2. Сделайте коммит с осмысленным сообщением:

```
bash
```

```
git commit -m "feat: Add event-driven server, logger, order handler, and event loop demo"
```

8.3. Отправьте ветку в удалённый репозиторий:

```
bash
```

```
git push -u origin feature/event-loop-tasks
```

8.4. Создайте Pull Request:

- Перейдите на GitHub в ваш репозиторий.
- Нажмите "Compare & pull request" (или создайте вручную через вкладку Pull requests).
- В заголовке укажите: "ЛР №12: Цикл событий и EventEmitter"
- В описании перечислите выполненные задания (можно скопировать список из пунктов 3-7).
- Отправьте PR на проверку преподавателю.